

Lung Cancer Database Documentation

Datasets Selected:

1. data_clinical_patient.txt
2. data_clinical_sample.txt
3. data_mutations.txt
4. data_cna.txt
5. data_mrna_seq_v2_rsem.txt
6. data_rppa.txt

These six databases were chosen because collectively they contain the essential information required to narrate a biological story on lung cancer. These databases contain clinical information on a per patient level, tumor information on a per sample level, information regarding DNA mutations, copy number alterations, mRNA expressions, and protein expressions.

The revised database design has been made more normalized compared to the initial database design. These design changes include the separation of information regarding cancer types into a separate lookup table, the separation of information concerning mutations into genomic variant information and annotations, storing of observations of mutations within a bridge table, and the placement of age in the clinical table since it is contextual.

Dataset Size Summary

1. data_clinical_patient.txt
Rows: 223
Columns: 15
Decision: Include fully, using clinically meaningful fields
2. data_clinical_sample.txt
Rows: 230
Columns: 10
Decision: Include fully
3. data_mutations.txt
Rows: 72,566
Columns: 250
Decision: Include mutation data, but only the main biologically useful columns
4. data_cna.txt
Rows: 23,423 genes
Columns: 232
Samples: 230
Decision: Include all genes and all samples

5. data_mrna_seq_v2_rsem.txt
Rows: 20,466 genes
Columns: 232
Samples: 230
Decision: Include all genes and all samples
6. data_rppa.txt
Rows: 160 protein markers
Columns: 182
Samples: 181
Decision: Include fully

Datasets Exploration:

This is a script for preliminary analysis of the TCGA LUAD (Lung Adenocarcinoma) dataset from cBioPortal. Six raw files will be analyzed and generate decision support outputs on database creation.

These will involve the following processes: load six files (Clinical Patient/Sample data, Mutations, CNA, mRNA, RPPA), create a series of outputs including column-wise missingness reports and keep/discard recommendation, samples and genes common across the different datasets, mutation details (top mutated genes, types, impact levels), occurrence of driver mutations such as EGFR, KRAS, among others, and a suggested schema for database tables.

The output will be saved in the form of a zip containing several CSV files and a decision document to be used as guidance prior to actual coding with SQL.

Overall Dataset Decision

All six datasets will be used for the database design. The clinical patient dataset, clinical samples dataset, CNA dataset, mRNA dataset, and RPPA dataset will be completely used. The mutations dataset will also be used, but all 250 columns of it will not be used. The only used columns will include those describing variant, genes, samples, alleles, variants classification, consequences, transcripts, and impacts.

The other significant decision is that the CNA and mRNA expression will not be filtered by genes. Even though they will result in a few million rows upon converting them into the relational model, this situation can be accepted since my aim is to create a good and complete database rather than to reduce rows.

The final schema will conform to 1NF, 2NF, 3NF, as well as BCNF and 4NF rules wherever possible. Also, the mutation model conforms to the principles of 5NF decomposition since raw facts about variants, annotation facts, and observations about samples are separated from each

other. Nevertheless, I do not claim strict compliance with 5NF since it requires proving all join dependencies and business rules.

Clinical Patient Dataset Decisions:

Dataset:

data_clinical_patient.txt

Decision:

Include this dataset fully, using the clinically meaningful fields.

Main fields to include:

- PATIENT_ID
- SEX
- AGE
- HISTOLOGICAL_SUBTYPE
- PRETREATMENT_HISTORY
- PRIMARY_TUMOR_PATHOLOGIC_SPREAD
- PRIOR_DIAGNOSIS
- RESIDUAL_TUMOR
- TOBACCO_SMOKING_HISTORY_INDICATOR
- OS_STATUS
- OS_MONTHS

Fields to exclude:

- DFS_STATUS
- DFS_MONTHS

Reason:

Patient-level data are included in the clinical patient database. This is necessary since the database must be able to link the molecular attributes of tumors to patient information and outcomes. The PATIENT_ID column is important since it serves as the primary patient identifier. It is necessary to include SEX as patient demographic information. The AGE column can be found in the PATIENT_CLINICAL table rather than PATIENT_DEMOGRAPHIC since it is time-dependent and refers to the age at diagnosis/enrollment/clinical observation.

Diagnosis columns such as HISTOLOGICAL_SUBTYPE provide information about the patient's cancer history. OS_STATUS and OS_MONTHS are very useful since survival analysis can be performed using this data. DFS_STATUS and DFS_MONTHS should be removed since they are entirely missing and thus not providing any useful information.

Future tables populated from this dataset:

- PATIENT
 - PATIENT_DEMOGRAPHIC
 - PATIENT_CLINICAL
 - PATIENT_SURVIVAL
 - PATIENT_SMOKING_HISTORY
-

Clinical Sample Dataset Decision

Dataset:

data_clinical_sample.txt

Decision:

Include this dataset fully.

Main fields to include:

- PATIENT_ID
- SAMPLE_ID
- CANCER_TYPE
- CANCER_TYPE_DETAILED
- ONCOTREE_CODE
- SAMPLE_TYPE
- SOMATIC_STATUS
- TMB_NONSYNONYMOUS
- TUMOR_STAGE_2009
- DISTANT_METASTASIS_PATHOLOGIC_SPREAD

Reason:

The clinical sample dataset is vital since molecular data is derived from a sample. Mutation, CNA, mRNA, and RPPA datasets are related to tumor samples, hence, the need for

SAMPLE_ID to tie all these molecular datasets to the sample table. Similarly, PATIENT_ID is needed to relate each sample to its respective patient.

In the last normalized schema, cancer types are segregated in the CANCER_TYPE lookup table. This implies that CANCER_TYPE, CANCER_TYPE_DETAILED, and ONCOTREE_CODE attributes will not appear in the SAMPLE table directly. Instead, CANCER_TYPE_ID will be used as a foreign key in SAMPLE to avoid repetition and eliminate transitive dependency.

TMB_NONSYNONYMOUS attribute is useful due to its capacity to compare tumor mutation burden among different mutation groups. Attributes like TUMOR_STAGE_2009 and DISTANT_METASTASIS_PATHOLOGIC_SPREAD can be useful in comparing molecular alterations and tumor stage/metastasis spread.

Future tables populated from this dataset:

- CANCER_TYPE
 - SAMPLE
 - SAMPLE_STAGE
 - SAMPLE_METRIC
-

Mutation Dataset Decision

Dataset:

data_mutations.txt

Decision:

Include mutation records, but only selected biologically important columns.

Dataset size:

72,566 mutation records
250 columns

Reason:

The mutation file is arguably the most essential dataset, as it provides somatic variants in lung cancer patients. Nevertheless, there are various other columns in this dataset that do not contribute to building the fundamental database. All columns are not necessary, as including all 250 columns would lead to an overly complicated database. The objective is to include only

those columns that characterize the variant itself, its corresponding gene, the impacted sample, the allelic change, and biological consequences.

Previously, all the mutation data had been included in a single MUTATION table, which was related to genes through the MUTATION_GENE table. This issue was resolved in the finalized version by segregating the mutation data into three different tables:

- GENOMIC_VARIANT
- VARIANT_ANNOTATION
- SAMPLE_MUTATION

The decision was made based on the notion that raw variant positions and alleles, biological annotations, and sample mutations constitute different facts.

Mutation columns to keep:

- HUGO_SYMBOL
- ENTREZ_GENE_ID
- CHROMOSOME
- START_POSITION
- END_POSITION
- CONSEQUENCE
- VARIANT_CLASSIFICATION
- VARIANT_TYPE
- REFERENCE_ALLELE
- TUMOR_SEQ_ALLELE1
- TUMOR_SEQ_ALLELE2
- TUMOR_SAMPLE_BARCODE
- T_REF_COUNT
- T_ALT_COUNT
- HGVSC
- HGVSP
- HGVSP_SHORT
- TRANSCRIPT_ID
- PROTEIN_POSITION
- CODONS
- HOTSPOT
- IMPACT

Mutation columns to exclude from the main database:

- COSMIC annotation fields
- EXAC frequency fields
- GO annotation fields

- DrugBank fields
- UniProt duplicate fields
- Extra duplicated transcript/protein annotation fields
- Other annotation-heavy fields not needed for the core relational model

Reason for excluding extra mutation columns:

These categories can be seen as external annotation fields or redundant metadata fields. These could be helpful during downstream annotation work, but there is no need to keep them in the primary normalized database table. Retaining only the relevant mutation fields helps maintain simplicity and biological significance.

Variant classification decision:

Keep VARIANT_CLASSIFICATION.

Reason:

VARIANT_CLASSIFICATION helps distinguish mutation types such as Missense_Mutation, Silent, Nonsense_Mutation, Frame_Shift_Del, Splice_Site, RNA, and Frame_Shift_Ins. This is biologically useful because some mutation classes are more likely to affect protein function than others.

Variant type decision:

Keep VARIANT_TYPE.

Reason:

VARIANT_TYPE describes the broad mutation category, such as SNP, DEL, INS, DNP, or TNP. This helps describe the mutation landscape of the tumor samples.

Mutation impact decision:

Keep IMPACT.

Reason:

IMPACT helps prioritize mutations by predicted biological effect. For example, HIGH and MODERATE impact mutations may be more biologically meaningful than LOW or MODIFIER mutations.

Genomic variant decision:

Create a separate GENOMIC_VARIANT table.

Reason:

The columns CHROMOSOME, START_POSITION, END_POSITION, REFERENCE_ALLELE, TUMOR_SEQ_ALLELE1, and TUMOR_SEQ_ALLELE2 describe the raw genomic variant. They should be stored independently of any biological annotation because a single genomic variation can have more than one biological annotation.

Variant annotation decision:

Create a separate VARIANT_ANNOTATION table.

Reason:

One single genetic mutation may have several interpretations, particularly when considering multiple transcripts or different genes. Fields such as VARIANT_CLASSIFICATION, VARIANT_TYPE, CONSEQUENCE, IMPACT, HGVSC, HGVSP, TRANSCRIPT_ID, PROTEIN_POSITION, CODONS, and HOTSPOT belong in the annotation table because they describe interpretation rather than the raw genomic position.

Sample mutation decision:

Keep sample-specific mutation observations in SAMPLE_MUTATION.

Reason:

T_REF_COUNT and T_ALT_COUNT are sample-specific. The same variant may appear in multiple samples, but with different read counts. Therefore, these fields should not be stored in GENOMIC_VARIANT; they belong in the bridge table that connects samples to variants.

Future tables populated from this dataset:

- GENE
- GENOMIC_VARIANT
- VARIANT_ANNOTATION
- SAMPLE_MUTATION

CNA Dataset Decision

Dataset:

data_cna.txt

Decision:

Include all genes and all samples.

Dataset size:

23,423 genes

230 samples

Reason:

The CNA dataset provides copy-number alteration values for each gene and sample. Copy-number alteration is an important genomic layer because it identifies gene amplifications and deletions. Since the project goal is to build a complete structure and row count is acceptable, all genes and all samples will be included.

CNA matrix format:

In the original file, rows represent genes and columns represent samples. Each cell contains a CNA value.

Future table populated from this dataset:

- COPY_NUMBER_ALTERATION
-

mRNA Expression Dataset Decision

Dataset:

data_mrna_seq_v2_rsem.txt

Decision:

Include all genes and all samples.

Dataset size:

20,466 genes

230 samples

Approximately 4,707,180 long-format records after conversion

Reason:

The mRNA dataset provides transcript-level gene expression values. Including all genes allows the database to support broader biological questions beyond only known driver genes. Since the focus is on building a strong structure and the row count is acceptable, all mRNA genes and samples will be included.

mRNA matrix format:

In the original file, rows represent genes and columns represent samples. Each cell contains an RSEM expression value.

Future table populated from this dataset:

- MRNA_EXPRESSION
-

RPPA Protein Expression Dataset Decision

Dataset:

data_rppa.txt

Decision:

Include the full RPPA dataset.

Dataset size:

160 protein markers

181 samples

Approximately 28,960 long-format records after conversion

Reason:

RPPA is small compared with CNA and mRNA, so it can be included fully. RPPA adds protein-level biological information, which is valuable because protein expression can provide functional evidence beyond DNA mutation and mRNA expression.

Cleaning decision:

RPPA identifiers may use a combined gene/protein or antibody format such as:

EGFR|EGFR-R-V
TP53|P53-R-V
STK11|LKB1-M-C

These should be parsed so that the gene symbol is taken from the part before the pipe symbol and the antibody/protein marker is taken from the part after the pipe symbol.

Reason:

Parsing RPPA identifiers correctly allows protein expression records to link to the central GENE table. Without this parsing, RPPA gene names may not match mutation, CNA, or mRNA gene names.

Future table populated from this dataset:

- PROTEIN_EXPRESSION
-

Final Table Plan

1. PATIENT

Source:

data_clinical_patient.txt

Purpose:

Stores one row per patient.

Main columns:

- PATIENT_ID
- PATIENT_BARCODE

Reason:

This is the central patient-level table. It avoids repeating patient identifiers across clinical, survival, smoking, demographic, and sample tables.

2. PATIENT_DEMOGRAPHIC

Source:

data_clinical_patient.txt

Purpose:

Stores static patient demographic information.

Main columns:

- DEMOGRAPHIC_ID
- PATIENT_ID
- SEX

Reason:

Demographic information belongs at the patient level. AGE was intentionally moved out of this table because age changes over time and is better treated as a clinical-context value, such as age at diagnosis or enrollment. Separating demographic information from the main patient table keeps the design organized and normalized.

Normalization decision:

This table uses a unique constraint on PATIENT_ID, meaning the design assumes one demographic record per patient.

3. PATIENT_CLINICAL

Source:

data_clinical_patient.txt

Purpose:

Stores general diagnosis and clinical background information.

Main columns:

- CLINICAL_ID
- PATIENT_ID
- AGE
- HISTOLOGICAL_SUBTYPE
- PRETREATMENT_HISTORY
- PRIMARY_TUMOR_PATHOLOGIC_SPREAD
- PRIOR_DIAGNOSIS
- RESIDUAL_TUMOR

Reason:

These fields describe the patient's cancer diagnosis and clinical background. They are separated from survival and sample-specific fields because they represent a different clinical concept. AGE is stored here because it is time-contextual and should represent the patient's age at diagnosis, enrollment, or clinical observation.

Normalization decision:

This table uses a unique constraint on PATIENT_ID, meaning the design assumes one clinical record per patient for this TCGA-style dataset. If longitudinal clinical records were needed later, the unique constraint could be removed and a clinical date column could be added.

4. PATIENT_SURVIVAL

Source:

data_clinical_patient.txt

Purpose:

Stores survival outcome information.

Main columns:

- SURVIVAL_ID
- PATIENT_ID
- OS_STATUS
- OS_MONTHS

Reason:

Overall survival is useful for biological interpretation and allows mutation or expression groups to be compared with patient outcomes.

Normalization decision:

This table uses a unique constraint on PATIENT_ID, meaning the design assumes one survival outcome record per patient. If longitudinal follow-up records were required, the unique constraint could be removed and a follow-up date column could be added.

5. PATIENT_SMOKING_HISTORY

Source:

data_clinical_patient.txt

Purpose:

Stores smoking history information.

Main columns:

- SMOKING_ID
- PATIENT_ID
- TOBACCO_SMOKING_HISTORY_INDICATOR

Reason:

Smoking is highly relevant in lung cancer. Keeping it in its own table allows it to be analyzed independently from other clinical fields. It also supports a more flexible model if smoking history ever needs to store more than one record per patient.

6. CANCER_TYPE

Source:

data_clinical_sample.txt

Purpose:

Stores cancer type lookup information.

Main columns:

- CANCER_TYPE_ID
- ONCOTREE_CODE
- CANCER_TYPE
- CANCER_TYPE_DETAILED

Reason:

Cancer type information can repeat across many samples. Moving ONCOTREE_CODE, CANCER_TYPE, and CANCER_TYPE_DETAILED into a lookup table avoids repeated values in the SAMPLE table and removes transitive dependency.

Normalization decision:

This table was added to satisfy 3NF more clearly. Instead of storing cancer type descriptions directly in SAMPLE, the sample table stores only CANCER_TYPE_ID.

7. SAMPLE

Source:

data_clinical_sample.txt

Purpose:

Stores one row per tumor sample.

Main columns:

- SAMPLE_ID
- SAMPLE_BARCODE
- PATIENT_ID
- CANCER_TYPE_ID
- SOMATIC_STATUS

Reason:

Molecular data is sample-level, not patient-level. This table connects each tumor sample back to the patient. Cancer type information is referenced using CANCER_TYPE_ID instead of repeating cancer type text fields directly in the sample table.

Normalization decision:

CANCER_TYPE, CANCER_TYPE_DETAILED, and ONCOTREE_CODE were removed from SAMPLE and placed in the CANCER_TYPE lookup table to remove transitive dependency.

8. SAMPLE_STAGE

Source:

data_clinical_sample.txt

Purpose:

Stores tumor staging and metastasis information.

Main columns:

- STAGE_ID
- SAMPLE_ID
- TUMOR_STAGE_2009
- DISTANT_METASTASIS_PATHOLOGIC_SPREAD

Reason:

Tumor stage and metastasis describe disease severity and are useful for comparing molecular alterations with clinical progression. These fields are separated from the basic SAMPLE table because they represent staging information rather than core sample identity.

Normalization decision:

This table uses a unique constraint on SAMPLE_ID, meaning the design assumes one pathologic staging record per sample. If re-staging over time were needed, the unique constraint could be removed and a staging date column could be added.

9. SAMPLE_METRIC

Source:

data_clinical_sample.txt

Purpose:

Stores sample-level quantitative metrics.

Main columns:

- SAMPLE_METRIC_ID
- SAMPLE_ID
- TMB_NONSYNONYMOUS

Reason:

Tumor mutation burden is sample-specific. It should be stored separately from basic sample identifiers and separately from staging information because it represents a genomic/sample metric, not clinical staging.

Normalization decision:

This table uses a unique constraint on SAMPLE_ID, meaning the design assumes one metric record per sample.

10. GENE

Sources:

data_mutations.txt
data_cna.txt
data_mrna_seq_v2_rsem.txt
data_rppa.txt

Purpose:

Stores one row per gene.

Main columns:

- GENE_ID
- HUGO_SYMBOL
- ENTREZ_GENE_ID

Reason:

Genes appear in multiple datasets. A central gene table prevents repeated gene information in mutation annotations, CNA, mRNA, and protein tables. Natural gene identifiers such as HUGO_SYMBOL are protected with a unique constraint, while GENE_ID is used as a surrogate primary key to simplify joins.

11. GENOMIC_VARIANT

Source:

data_mutations.txt

Purpose:

Stores raw genomic variant position and allele information.

Main columns:

- VARIANT_ID
- CHROMOSOME
- START_POSITION
- END_POSITION
- REFERENCE_ALLELE
- TUMOR_SEQ_ALLELE1
- TUMOR_SEQ_ALLELE2

- VARIANT_HASH

Reason:

This table describes the raw genomic variant itself. It stores the chromosome, position, reference allele, and tumor alternate alleles. These fields were separated from annotation fields because the same raw variant can have multiple annotations.

Normalization decision:

The previous MUTATION table was split. Raw position and allele facts are stored in GENOMIC_VARIANT, while biological interpretation is stored in VARIANT_ANNOTATION. This avoids repeating the same genomic position and allele values for every transcript or annotation.

Practical database decision:

VARIANT_HASH is a generated technical column used to enforce uniqueness across chromosome, start position, end position, reference allele, and alternate alleles. This was used instead of a large unique key over multiple TEXT columns because MySQL can have index length limitations. The hash does not violate normalization because it is generated from existing fields and does not introduce new biological information.

12. VARIANT_ANNOTATION

Source:

data_mutations.txt

Purpose:

Stores biological annotation and interpretation of genomic variants.

Main columns:

- ANNOTATION_ID
- VARIANT_ID
- GENE_ID
- VARIANT_CLASSIFICATION
- VARIANT_TYPE
- CONSEQUENCE
- IMPACT

- HGVSC
- HGVSP
- HGVSP_SHORT
- TRANSCRIPT_ID
- PROTEIN_POSITION
- CODONS
- HOTSPOT

Reason:

This table stores the biological meaning of a genomic variant. A single raw variant can have multiple annotations depending on gene, transcript, and consequence. Keeping these fields outside of GENOMIC_VARIANT avoids redundancy and supports a more normalized structure.

Normalization decision:

This table replaces the previous MUTATION_GENE design. Instead of using a separate mutation-gene bridge table, VARIANT_ANNOTATION links each variant to the affected gene and stores the annotation details in the same annotation record. This is more appropriate because the gene relationship and the annotation facts are connected.

13. SAMPLE_MUTATION

Source:

data_mutations.txt

Purpose:

Connects samples to genomic variants.

Main columns:

- SAMPLE_MUTATION_ID
- SAMPLE_ID
- VARIANT_ID
- T_REF_COUNT
- T_ALT_COUNT

Reason:

One sample can have many variants, and the same variant can appear in many samples. This bridge table correctly models the many-to-many relationship between samples and genomic variants.

T_REF_COUNT and T_ALT_COUNT are stored here because they are sample-specific observations. They depend on the sample-variant relationship, not on the variant alone.

Normalization decision:

This table preserves the many-to-many relationship between samples and variants and prevents storing sample-specific read counts in the raw variant table.

14. COPY_NUMBER_ALTERATION

Source:

data_cna.txt

Purpose:

Stores CNA values for every gene-sample pair.

Main columns:

- CNA_ID
- SAMPLE_ID
- GENE_ID
- CNA_VALUE

Decision:

Include all genes and all samples from the CNA file.

Reason:

Copy-number alteration is an important genomic layer. Since row count is acceptable, the full CNA matrix should be converted into relational format. The CNA_VALUE depends on the combination of sample and gene, not on sample alone or gene alone.

Normalization decision:

A unique constraint on SAMPLE_ID and GENE_ID prevents duplicate CNA values for the same sample-gene pair.

15. MRNA_EXPRESSION

Source:

data_mrna_seq_v2_rsem.txt

Purpose:

Stores mRNA expression values for every gene-sample pair.

Main columns:

- MRNA_ID
- SAMPLE_ID
- GENE_ID
- RSEM_VALUE

Decision:

Include all genes and all samples from the mRNA file.

Reason:

mRNA expression provides transcript-level biological information and allows broader analysis across all genes. The RSEM_VALUE depends on the combination of sample and gene.

Normalization decision:

A unique constraint on SAMPLE_ID and GENE_ID prevents duplicate mRNA expression values for the same sample-gene pair.

16. PROTEIN_EXPRESSION

Source:

data_rppa.txt

Purpose:

Stores RPPA protein expression values.

Main columns:

- PROTEIN_EXPRESSION_ID
- SAMPLE_ID
- GENE_ID
- ANTIBODY_REF
- EXPRESSION_VALUE

Decision:

Include all RPPA protein records.

Reason:

RPPA is small enough to include fully and provides protein-level biological interpretation. Protein expression depends on the sample, gene, and antibody/protein marker.

Normalization decision:

A unique constraint on SAMPLE_ID, GENE_ID, and ANTIBODY_REF is used because the same gene and sample may have different protein expression measurements depending on the antibody or marker used.

Overall Normalization Decisions

1NF Justification

The design is 1NF-compliant since all fields have atomic values and no repeating groups exist within each row. The matrix files, CNA, mRNA, and RPPA, are transformed to long form for each observation as a unique row.

2NF Justification

The design is 2NF-compliant since the non-key attribute is dependent upon either the whole key or the whole relationship depicted by the table. For instance, RSEM_VALUE relies on sample

and gene, CNA_VALUE relies on sample and gene, while read counts rely on sample and variant.

3NF Justification

The design is 3NF-compliant since any transitive dependencies are eliminated. CANCER_TYPE includes cancer information and does not store them within SAMPLE, GENE has gene details and does not keep them in molecular observations tables. FACT_PATIENT, FACT_SAMPLE, FACT_GENE, and FACT_MUTATION contain patient facts, sample facts, gene facts, and mutation facts respectively without any interrelationships.

BCNF and 4NF Justification

The schema is almost BCNF/4NF-compliant as all determinant attributes are either the candidate or foreign keys and multivalued facts exist in distinct tables. For instance, sample mutations, variant annotations, mRNA expression, protein expression, CNA, smoking status, and survival facts do not co-exist in the same table.

5NF-Style Decision

The mutation model follows 5NF-style decomposition because raw variant information, biological annotation, and sample-specific variant observations are separated. However, the database does not claim full formal 5NF because that would require proving all join dependencies and business rules.

Slight Deviations and Practical Design Decisions

Some practical modeling decisions were made while still preserving normalization:

- Surrogate primary keys such as PATIENT_ID, SAMPLE_ID, GENE_ID, VARIANT_ID, and ANNOTATION_ID were used to simplify joins.
- Natural identifiers such as PATIENT_BARCODE, SAMPLE_BARCODE, HUGO_SYMBOL, and ONCOTREE_CODE were still protected with unique constraints.
- One-to-one child tables such as PATIENT_DEMOGRAPHIC, PATIENT_CLINICAL, PATIENT_SURVIVAL, SAMPLE_STAGE, and SAMPLE_METRIC were kept separate because they represent different categories of facts.

Summary of Normalization Process:

The ultimate database design involved the application of normalization concepts together with practical modeling approaches, which were based on the structural characteristics of TCGA-style cancer data. Information about the type of cancer was placed in a lookup table in order to eliminate transitive dependency. The ID of the patient was removed from clinical,

demographics, smoking, and survival attributes. Attributes related to the level of sample were separated from those related to the level of patient since one patient could have several samples. Information about genes was stored in a reference table just once and then reused through foreign keys. Mutation information was broken down into variant facts, biological annotation facts, and mutations in samples. Information on gene expression, protein expression, CNA, sample staging, and sample metrics were stored in individual tables as each table had different dependencies. Several deviations that occurred intentionally included the usage of one-to-one child tables for patient and sample facts, surrogate key, and finally, hash value for enforcing the uniqueness of variants in MySQL.

Database Cleaning Steps:

Missing Value Standardization

All common representations of missing data are normalized to NaN — this covers strings like "NA", "N/A", "NULL", "[Not Available]", "Not Available", "Unknown", "Not Reported", "Not Applicable", blank strings, and whitespace-only strings. This is applied on initial file load and again after column extraction.

Column Name Normalization:

Every column name is stripped of leading/trailing whitespace, spaces and hyphens and slashes are replaced with underscores, all non-alphanumeric characters are removed, and the result is uppercased. Duplicate column names get a numeric suffix to avoid collisions.

Sample Barcode Standardization

All sample barcodes have underscores replaced with hyphens to enforce TCGA barcode formatting consistency across files.

Patient Barcode Derivation

When a patient ID isn't directly available in the sample file, it's derived by taking the first 12 characters of the TCGA sample barcode, which is the standard TCGA patient identifier prefix.

Gene Symbol Standardization

Gene symbols are stripped of whitespace, uppercased, and pipe-delimited entries (e.g. GENE|12345) are split so only the symbol is kept. Empty strings are converted to NaN.

RPPA Feature Parsing

RPPA row identifiers in the format GENE|ANTIBODY are split into two separate fields — HUGO_SYMBOL and ANTIBODY_REF — so the data can be properly normalized.

Numeric Type Coercion

Columns that should be numeric (AGE, OS_MONTHS, START_POSITION, END_POSITION, T_REF_COUNT, T_ALT_COUNT, ENTREZ_GENE_ID, TMB, CNA values, RSEM values, RPPA expression values) are all coerced with `pd.to_numeric(..., errors="coerce")`, turning unparseable strings into NaN rather than raising errors.

Deduplication

- Clinical patient rows are deduplicated on PATIENT_BARCODE
- Clinical sample rows are deduplicated on SAMPLE_BARCODE
- Genomic variants are deduplicated on VARIANT_HASH
- Gene lists from all sources are merged and deduplicated on HUGO_SYMBOL, with preference given to rows that have an Entrez gene ID
- Required Field Filtering (Row Dropping)

Rows missing critical identifiers are dropped entirely:

- Clinical patient: drops if PATIENT_BARCODE is null
- Clinical sample: drops if either SAMPLE_BARCODE or PATIENT_BARCODE is null
- Mutations: drops if any of HUGO_SYMBOL, CHROMOSOME, START_POSITION, END_POSITION, or TUMOR_SAMPLE_BARCODE is null
- Matrix long-format files: drop rows where SAMPLE_BARCODE, HUGO_SYMBOL, or the value column is null

Script Map: Script Execution Order

Run all scripts in the order listed below. Do not skip steps or change the order, as each script depends on the outputs of the previous one.

Step 1 — Data Exploration

File: ``database_exploration_cleaning.py`` (Part 1: Exploration)

Environment: Google Colab

Input: Six raw `.txt`` files from cBioPortal in Google Drive

Output: Decision-support CSVs and a written report (zipped and downloaded)

What it does:

- Loads all six raw cBioPortal files
- Generates column-by-column missingness reports

- Summarizes sample and gene overlap across datasets
- Identifies top mutated genes and variant classifications
- Checks presence of known LUAD driver genes (EGFR, KRAS, TP53, etc.)
- Produces a recommended table plan and written decision report

Step 2 — Data Cleaning

File: `database\_exploration\_cleaning.py` (Part 2: Cleaning)

Environment: Google Colab

Input: Six raw `.txt` files from cBioPortal in Google Drive

Output: Cleaned CSV staging files (zipped and downloaded)

What it does:

- Standardizes all missing values to NaN
- Normalizes column names to uppercase with underscores
- Standardizes sample barcodes (underscores replaced with hyphens)
- Derives patient barcodes from sample barcodes where needed
- Standardizes gene symbols to uppercase
- Parses RPPA identifiers into HUGO_SYMBOL and ANTIBODY_REF
- Coerces numeric fields to correct types
- Deduplicates patients, samples, variants, and genes
- Converts CNA, mRNA, and RPPA matrices from wide to long format
- Splits mutation data into three staging files:
 - `genomic\_variant\_source.csv` — raw variant positions and alleles
 - `variant\_annotation\_source.csv` — biological annotation
 - `sample\_mutation\_source.csv` — sample-specific read counts
- Generates VARIANT_HASH for each unique genomic variant
- Produces a validation report of row counts and overlap checks

Step 3 — SQL File Generation

File: `Database\_loadsql.py`

Environment: Local Python (run on your machine)

Input: Cleaned CSV staging files from Step 2

Output: Numbered SQL files ready for import into MySQL

What it does:

- Reads all cleaned CSV files
- Generates INSERT statements for all 16 tables
- Resolves foreign keys using subquery lookups on natural identifiers
- Splits large tables (CNA, mRNA, RPPA) into 4 part files each to avoid phpMyAdmin timeouts
- Generates a master load file and a validation counts file

Step 4 — Schema Creation

File: `01\_create\_schema.sql`

Environment: MySQL / phpMyAdmin

Input: None

Output: All 16 empty tables with primary keys, foreign keys, and constraints

Run this before any data is loaded.

Step 5 — Data Loading

Environment: MySQL / phpMyAdmin

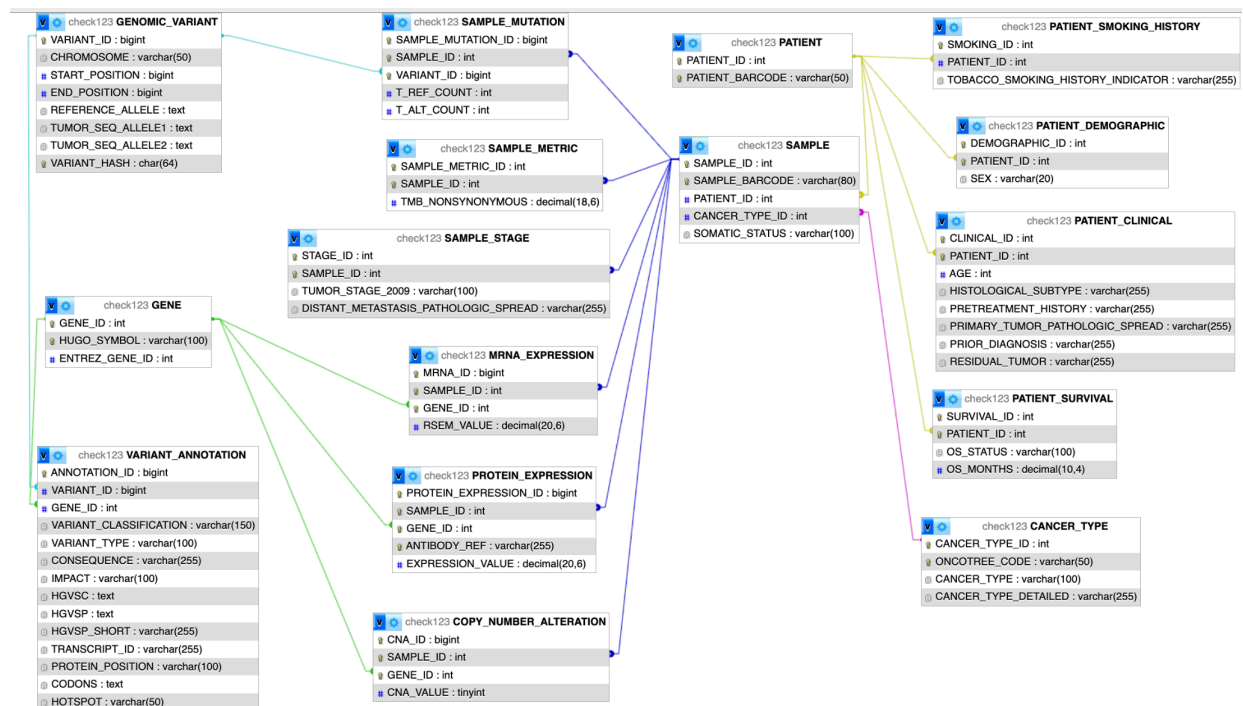
Input: SQL files generated in Step 3

Import each SQL file in the order shown in Step 3.

For the large part files (CNA, mRNA, RPPA), import one part at a time.

If a part times out, rerunning it is safe because all inserts use INSERT IGNORE.

Database Structure:



SQL Select Statement:

```
SELECT g.HUGO_SYMBOL, va.VARIANT_CLASSIFICATION, va.IMPACT, COUNT(DISTINCT sm.SAMPLE_ID) AS SAMPLES_MUTATED FROM GENE g JOIN VARIANT_ANNOTATION va ON va.GENE_ID = g.GENE_ID JOIN GENOMIC_VARIANT gv ON gv.VARIANT_ID = va.VARIANT_ID JOIN SAMPLE_MUTATION sm ON sm.VARIANT_ID = gv.VARIANT_ID GROUP BY g.HUGO_SYMBOL, va.VARIANT_CLASSIFICATION, va.IMPACT HAVING SAMPLES_MUTATED >= 10 ORDER BY SAMPLES_MUTATED DESC LIMIT 50;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Extra options

HUGO_SYMBOL	VARIANT_CLASSIFICATION	IMPACT	SAMPLES_MUTATED
TTN	Missense_Mutation	MODERATE	98
MUC16	Missense_Mutation	MODERATE	85
KRAS	Missense_Mutation	MODERATE	72
CSDM3	Missense_Mutation	MODERATE	69
RYR2	Missense_Mutation	MODERATE	69
TP53	Missense_Mutation	MODERATE	67
USH2A	Missense_Mutation	MODERATE	61
LRP1B	Missense_Mutation	MODERATE	60
FLG	Missense_Mutation	MODERATE	56
ZFXH4	Missense_Mutation	MODERATE	56
SPTA1	Missense_Mutation	MODERATE	50
MUC17	Missense_Mutation	MODERATE	45
TTN	Silent	LOW	45
KMT2C	Missense_Mutation	MODERATE	41
NAV3	Missense_Mutation	MODERATE	41
FAT3	Missense_Mutation	MODERATE	40
XIRP2	Missense_Mutation	MODERATE	40
PCDH15	Missense_Mutation	MODERATE	39
AHNAK2	Missense_Mutation	MODERATE	38
APOB	Missense_Mutation	MODERATE	38
PCLO	Missense_Mutation	MODERATE	38
CSDM1	Missense_Mutation	MODERATE	36
MUC16	Silent	LOW	35
ZNF536	Missense_Mutation	MODERATE	35
ANK2	Missense_Mutation	MODERATE	34
CDH10	Missense_Mutation	MODERATE	34
RP1L1	Missense_Mutation	MODERATE	34
RYR1	Missense_Mutation	MODERATE	34
GPR112	Missense_Mutation	MODERATE	33
KEAP1	Missense_Mutation	MODERATE	33
PAPPA2	Missense_Mutation	MODERATE	33
ASXL3	Missense_Mutation	MODERATE	32
COL11A1	Missense_Mutation	MODERATE	32
FLG2	Missense_Mutation	MODERATE	32
SI	Missense_Mutation	MODERATE	32
Console	Missense_Mutation	MODERATE	31

Limitations:

DFS_STATUS and DFS_MONTHS are missing

Disease-free survival fields were completely absent in the source data and were excluded. Survival analysis is limited to overall survival (OS_STATUS, OS_MONTHS).

228 of 250 mutation columns were excluded

The mutation source file contains external annotation fields from COSMIC, ExAC, GO, DrugBank, and UniProt. These were excluded to keep the schema focused on core biological fields. They could be added in a future extension table if needed.

TTN appears as a top mutated gene but is likely a false positive

TTN is one of the largest genes in the human genome and accumulates mutations by chance rather than because of biological selection. It consistently appears at the top of mutation frequency lists in cancer genomics and should be interpreted with caution.

Schema assumes one record per patient per clinical category

PATIENT_CLINICAL, PATIENT_SURVIVAL, PATIENT_DEMOGRAPHIC, and PATIENT_SMOKING_HISTORY

each have a unique constraint on PATIENT_ID. This is appropriate for a cross-sectional TCGA-style dataset but would require constraint changes and date fields if longitudinal follow-up records were needed.

RPPA covers fewer samples than CNA and mRNA

The RPPA file contains 181 samples compared to 230 samples in the CNA and mRNA files. Not all samples have protein expression data. Queries joining PROTEIN_EXPRESSION with other molecular tables will return fewer rows than equivalent CNA or mRNA joins.

Full formal 5NF is not claimed

The mutation model follows 5NF-style decomposition in its separation of raw variant facts, biological annotations, and sample observations. However, strict 5NF compliance would require proving all join dependencies and business rules explicitly, which is beyond the scope of this project.